# ROAD2HIRE

# Backend Development & API Integration Report

Collaborator handoff document

| Project detail | Description |
|---|---|
| Project | Road2Hire — AI-Powered Career Readiness Platform |
| Purpose | Backend implementation, API development, and frontend–backend–AI integration |
| Backend | Python + FastAPI |
| Database | PostgreSQL (planned architecture; verify existing configuration) |
| Frontend | Next.js, React, TypeScript |
| Primary goal | Deliver reliable, documented APIs connecting the UI, database, and available AI/ML modules. |

**Scope note**
**This report includes proposed API contracts and implementation tasks. The collaborator should inspect the current repository and confirm existing routes and schemas before adding endpoints.**

Prepared for the Road2Hire backend collaborator

# 1. Project Overview

Road2Hire is an AI-powered career readiness platform that helps users upload and analyze resumes, extract skills, identify gaps for a target role, understand career readiness, receive personalized learning roadmaps, track progress, and obtain career guidance through an assistant.

The backend receives frontend requests, validates inputs, manages user and candidate data, coordinates AI/ML processing, persists results, and returns structured responses.

## System architecture

```
User
  |
Next.js Frontend
  |  HTTP / JSON / Multipart Upload
FastAPI Backend
  +-- Authentication & Authorization
  +-- Request Validation & Business Logic
  +-- PostgreSQL Database
  +-- Resume Processing Pipeline
  +-- Skill Matching & Skill-Gap Analysis
  +-- Career Readiness & Recommendations
  +-- Learning Roadmap & Progress
  +-- RAG Career Assistant
  |
Structured API Response
  |
Next.js Frontend
```

The frontend should not access the database or internal AI modules directly. Application data and AI operations should pass through backend APIs.

# 2. Current Backend Foundation

- **Backend/Main.py** — application entry point.
- **Backend/Api/auth.py** — authentication-related API module.
- **Backend/Core/database.py** — database configuration and connection/session logic.
- **Backend/Core/security.py** — security-related functionality.
- **Backend/Models/user.py** — user database model.
- **Backend/Schemas/user.py** — user request/response schemas.
- **Backend/alembic.ini**, **Backend/alembic/env.py** — migration configuration.
- **Backend/alembic/versions/a2ba68957958_create_users_table.py** — user-table migration.
- **Backend/Requirements.txt** — backend Python dependencies.

Important: The existence of these files does not confirm that every route, authentication flow, or database operation is complete. First inspect the code and run the current backend.

# 3. Backend Responsibilities

## API development

- Create and maintain FastAPI routes; define Pydantic request/response schemas.
- Validate incoming data and return consistent JSON responses and HTTP status codes.
- Expose API documentation through FastAPI/OpenAPI.

## Database management

- Configure PostgreSQL connectivity, models, relationships, and migrations.
- Store users, profiles, resume metadata, analysis results, and learning progress.
- Apply ownership checks so users cannot access another user's private resources.

### Authentication, security, AI integration, and frontend integration

- Implement safe password storage and agreed token/session handling; protect private routes.
- Validate uploaded files and keep secrets in environment variables.
- Connect available AI modules, validate outputs, persist supported results, and handle service failures.
- Provide the frontend collaborator with confirmed routes, schemas, base URL, CORS settings, and error conventions.

## 4. Proposed API Catalogue

Use /api/v1 as the proposed prefix unless the existing backend already follows another convention. These are proposed contracts, not a claim that all endpoints currently exist.

| Module | Method | Proposed endpoint | Purpose |
|---|---|---|---|
| Health | GET | /health | Check API status |
| Auth | POST | /api/v1/auth/register | Register a user |
| Auth | POST | /api/v1/auth/login | Authenticate a user |
| Profile | GET | /api/v1/profile | Retrieve current candidate profile |
| Profile | PATCH | /api/v1/profile | Update candidate information |
| Dashboard | GET | /api/v1/dashboard | Retrieve dashboard summary |
| Resume | POST | /api/v1/resumes | Upload/register a resume |
| Resume | GET | /api/v1/resumes | List current user's resumes |
| Resume | GET | /api/v1/resumes/{resume_id}/analysis | Retrieve analysis results |
| Resume | DELETE | /api/v1/resumes/{resume_id} | Delete resume, subject to retention rules |
| Skill gaps | GET | /api/v1/skill-gaps | Retrieve gaps for a target role |
| Readiness | GET | /api/v1/career-readiness | Retrieve readiness score/explanation |
| Roadmap | GET | /api/v1/learning-roadmap | Retrieve learning tasks |
| Roadmap | PATCH | /api/v1/tasks/{task_id} | Update task progress |
| Assistant | POST | /api/v1/assistant/messages | Submit a career question |
| Activity | GET | /api/v1/activity | Retrieve recent activity |
| Job matches | GET | /api/v1/jobs/matches | Retrieve suitable role/job matches |

Suggested order: health → authentication/profile → resume upload and analysis → dashboard → skill gaps/readiness → roadmap/progress → assistant/activity/job matching. Avoid duplicate routes when equivalent functionality already exists.

## 5. API Behaviour and Example Contracts

The following examples provide a common integration starting point. Exact fields and status values must be finalized against existing models and the AI extraction contract.

### 5.1 Health check

```
GET /health

{
  "status": "ok"
}
```

## 5.2 User registration

```
POST /api/v1/auth/register
Content-Type: application/json

{
  "name": "Demo Candidate",
  "email": "candidate@example.com",
  "password": "example-password"
}
```

- Validate required fields and email format; reject duplicate emails.
- Hash the password before storing it. Never return the password or its hash.
- Return a documented success response and suitable HTTP status.

## 5.3 User login

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "candidate@example.com",
  "password": "example-password"
}
```

The response should use the agreed authentication mechanism (for example, an access token) and return only safe user information. Document how the frontend sends credentials on subsequent requests.

## 5.4 Resume upload

```
POST /api/v1/resumes
Content-Type: multipart/form-data

file: candidate_resume.pdf
```

- Authenticate the user; validate file type, size, and content.
- Associate the resume with the current user and store it safely.
- Invoke the available resume-processing pipeline and validate its output.
- Persist analysis results and return a resume identifier plus processing status.

```
Illustrative response:
{
  "resume_id": "resume_123",
  "status": "completed",
  "message": "Resume processed successfully"
}
```

If processing is asynchronous, return a pending/accepted state and provide an endpoint to retrieve status and results. The example ID/status are illustrative.

## 5.5 Resume analysis

```
GET /api/v1/resumes/{resume_id}/analysis

{
  "resume_id": "resume_123",
  "status": "completed",
  "extracted_skills": [
    {"name": "Java", "source": "explicit", "confidence": 0.96},
    {"name": "SQL", "source": "explicit", "confidence": 0.91}
  ],
  "summary": {
    "education": [],
    "experience": [],
    "projects": []
  }
}
```

This response is illustrative, not the final extraction schema. Preserve schema versioning, evidence/provenance, confidence validation, and the distinction between explicit facts and inferred skills. Inferred skills must not become verified candidate facts.

## 5.6 Skill-gap analysis

```
GET /api/v1/skill-gaps?targetRole=Full%20Stack%20Developer

{
  "target_role": "Full Stack Developer",
  "matched_skills": ["JavaScript"],
  "missing_skills": ["Backend API Development", "Database Design"],
  "recommendations": [
    {
      "skill": "Backend API Development",
      "priority": "high",
      "reason": "Required for the selected target role"
    }
  ]
}
```

Matching and priority logic should come from the available AI/ML or business-logic implementation, not arbitrary hardcoded frontend assumptions.

## 5.7 Career readiness

```
GET /api/v1/career-readiness

{
  "target_role": "Full Stack Developer",
  "readiness_score": 68,
  "explanation": "Some required skills and project evidence are still missing.",
  "strengths": [],
  "improvement_areas": []
}
```

Define the score scale and calculation. Return scores and explanations only when supported by the underlying analysis.

## 5.8 Learning roadmap

```
GET /api/v1/learning-roadmap

{
  "target_role": "Full Stack Developer",
  "tasks": [
    {
      "id": "task_001",
      "title": "Build a REST API",
      "skill": "Backend API Development",
      "priority": "high",
      "status": "not_started"
    }
  ]
}
```

Tie roadmap tasks to the candidate's gaps and goal. Keep task identifiers stable between reads unless a new roadmap is intentionally generated.

## 5.9 Update task progress

```
PATCH /api/v1/tasks/{task_id}
Content-Type: application/json

{
  "status": "completed"
}
```

- Validate the status and verify the task belongs to the authenticated user.
- Persist the update and return the updated task.
- Avoid duplicate completion counts when the same task is updated repeatedly.

### 5.10 Career assistant

```
POST /api/v1/assistant/messages
Content-Type: application/json

{
  "message": "What should I learn next for a full-stack role?"
}

Illustrative response:
{
  "reply": "Start by identifying the highest-priority missing skills for your target role.",
  "sources": []
}
```

Use candidate context and the available career/learning knowledge base where supported. If RAG or an LLM service is unavailable, return a controlled error or clearly identified fallback; do not misrepresent fallback text as a genuine AI-generated result.

## 6. Standard API Error Handling

Use consistent status codes and a predictable error response across routes.

| Status | Meaning | Example |
|---|---|---|
| 200 | Successful request | Profile or analysis retrieved |
| 201 | Resource created | Registration or resume resource created |
| 202 | Accepted | Asynchronous resume processing pending |
| 400 | Invalid request | Malformed input |
| 401 | Unauthenticated | Missing/invalid credentials |
| 403 | Forbidden | Operation not permitted |
| 404 | Not found | Resource absent or inaccessible |
| 409 | Conflict | Duplicate email |
| 413 | Payload too large | Resume exceeds limit |
| 415 | Unsupported media type | Unsupported file type |
| 422 | Validation error | Invalid schema/field |
| 500 | Internal error | Unexpected server failure |
| 503 | Service unavailable | AI dependency unavailable |

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The supplied request is invalid",
    "details": []
  }
}
```

Choose one error structure and use it consistently. Never expose stack traces, secrets, raw database errors, or unnecessary sensitive resume content in public responses.

## 7. Database Responsibilities

PostgreSQL is the planned database architecture. Inspect existing database code and migrations before adding tables.

| Logical entity | Purpose |
|---|---|
| Users | Authentication and account information |
| Candidate profiles | Education, experience, preferences, career goal |
| Resumes | File metadata, ownership, processing status |

| Logical entity | Purpose |
|---|---|
| Resume analyses | Extracted facts, skills, evidence, analysis version |
| Skill gaps | Matched/missing skills for a target role |
| Career readiness | Scores, explanations, analysis references |
| Roadmaps / tasks | Learning plan, task state, stable IDs |
| Activity | Relevant user actions and progress |
| Assistant conversations | Conversation metadata/messages, if persistence is required |

- Confirm the database URL and environment variable names.
- Verify connection/session handling in `Backend/Core/database.py`.
- Confirm the user model and migration; add models and relationships incrementally.
- Create and test Alembic migrations.
- Filter private resources by user ownership; use transactions and appropriate constraints.
- Store uploaded files safely; avoid database blobs unless intentionally chosen.

# 8. Authentication and Security

- Password hashing and credential validation.
- Access token/session creation and verification.
- Protected-route dependencies and per-user ownership checks.
- Input validation and safe response serialization.
- Resume upload limits, content validation, and safe file handling.
- CORS restricted to approved frontend origins.
- Environment-based database credentials and secrets; no real secrets committed.
- Safe logs that omit passwords, tokens, and unnecessary personal data.

The frontend must not be trusted to enforce authorization by itself.

# 9. AI/ML Integration Contract

The backend connects the application APIs to the available AI/ML components. The repository includes these files; inspect their actual functions, inputs, outputs, and dependencies before integration:

- `AI/Resume_Processing/pdf_extractor.py`
- `AI/Resume_Processing/section_detector.py`
- `AI/Resume_Processing/resume_pipeline.py`
- `AI/Baseline/keyword_baseline.py`
- `AI/Knowledge_Graph/skill_graph_v1.json`

```
Resume Upload
  -> File Validation
  -> PDF/Text Extraction
  -> Section Detection
  -> Resume Pipeline
  -> Structured Output Validation
  -> Skill Normalization / Matching
  -> Skill-Gap Analysis
  -> Persist Validated Results
  -> API Response
```

- Define schemas for each module's input and output.
- Validate output before storing or returning it.
- Preserve explicit evidence versus inference.

- Handle unreadable resumes, empty results, and malformed outputs.
- Version extraction contracts and invalidate stale results after resume replacement.
- Return clear processing states and controlled errors.
- Keep model-specific code separate from route handlers where practical.

Do not assume career prediction, RAG, or recommendation components are complete just because related files exist. Integrate what is available and document remaining gaps.

# 10. Environment Configuration

Provide a `.env.example` with variable names and placeholders only. Confirm actual names against the code.

```
DATABASE_URL=
FRONTEND_ORIGIN=
SECRET_KEY=
ACCESS_TOKEN_EXPIRE_MINUTES=
MAX_RESUME_SIZE_MB=
```

- Never commit real secrets; keep local environment files out of Git.
- Document database creation, migrations, backend startup, and AI model paths.
- Configure the frontend base URL, for example `NEXT_PUBLIC_API_BASE_URL`, to match the actual setup.

# 11. Frontend–Backend Integration

## Backend collaborator should provide

- Confirmed routes and methods; request/response examples for implemented endpoints.
- Authentication mechanism and protected-route requirements.
- Backend base URL, CORS configuration, and environment variable names.
- Resume upload limits and processing states; standard error format.
- AI extraction contract and validation rules.
- API documentation URL (normally `/docs` in local development) and known limitations.

## Frontend collaborator will then

- Configure the base URL and central API client.
- Add TypeScript types matching confirmed schemas.
- Connect resume, dashboard/profile, skill-gap, readiness, roadmap, and task flows.
- Add loading, error, empty, and authentication states.
- Test against the actual backend; do not treat demo data as real analysis.

# 12. Backend Implementation Tasks and Priority

## Phase 1 — Audit and establish the API

- ■ Inspect `Backend/Main.py` and existing routers; identify already implemented routes.

Deliverable: working startup, route inventory, and setup instructions.

- ■ Run the backend; verify `/docs`, health endpoint, and database connectivity.
- ■ Document current routes and environment variables.

## Phase 2 — Authentication and candidate profile

- ■ Complete registration/login and verify password hashing plus token/session handling.
- ■ Protect private routes; implement profile retrieval/update.
- ■ Test ownership checks and database persistence.

Deliverable: Authenticated users can access only their own profile and resources.

### Phase 3 — Resume upload and analysis

- ■ Validate resume type, size, and content.
- ■ Integrate the available resume-processing pipeline.
- ■ Validate provenance/extraction results; store metadata and analysis.
- ■ Implement analysis retrieval, errors, replacement, stale-result handling, and deletion rules.

Deliverable: Resume upload, processing, and analysis retrieval work through documented APIs.

### Phase 4 — Career insights

- ■ Implement skill-gap retrieval for a target role and connect available matching logic.
- ■ Implement career-readiness responses and supported explanations.
- ■ Handle missing profiles and unavailable model results.

Deliverable: Frontend retrieves structured insights grounded in available analysis.

### Phase 5 — Roadmap and progress

- ■ Retrieve candidate-specific learning roadmaps with stable IDs.
- ■ Implement task status updates and persist progress.
- ■ Prevent duplicate completion counting and keep dashboard progress consistent.

Deliverable: Roadmap progress persists and stays consistent across views.

### Phase 6 — Assistant and remaining modules

- ■ Implement assistant messages and connect available retrieval/RAG functionality.
- ■ Define source metadata and fallback behaviour.
- ■ Implement activity and job matches when data and matching logic are available.

Deliverable: Implemented endpoints work with their services or expose documented pending/unavailable states.

### Phase 7 — Integration and documentation

- ■ Add unit/API tests for validation, business logic, success and failure cases.
- ■ Add resume-flow integration tests; test authentication and ownership restrictions.
- ■ Validate AI response contracts and document setup, schemas, errors, and limitations.
- ■ Provide the frontend collaborator with working API examples.

Deliverable: A documented, testable backend ready for frontend integration.

## 13. Testing Checklist

### API and database

- ■ Backend starts; health endpoint responds; database connection succeeds.
- ■ Migrations apply to a clean development database.
- ■ Invalid requests and missing resources return appropriate structured errors.

### Authentication and security

- ■ Duplicate registration and incorrect credentials are rejected.
- ■ Protected routes reject unauthenticated requests.
- ■ Users cannot access another user's resumes/tasks; secrets are not returned.

### Resume and AI processing

- ■ Valid upload succeeds; unsupported/oversized/unreadable files are handled safely.
- ■ Invalid/missing extraction fields follow the contract.
- ■ Inferred skills are not stored as verified facts; replacement invalidates stale analysis.

### Integration

- ■ Frontend base URL and CORS work as configured.
- ■ Resume upload/analysis succeeds end to end.
- ■ Dashboard and roadmap data are consistent; task updates persist.
- ■ Loading, error, empty, and unavailable-service states behave correctly.

## 14. Definition of Done

- ■ Backend starts using documented instructions and implemented routes appear in API docs.
- ■ Request/response schemas are documented and consistent.
- ■ Database persistence, authentication, and ownership checks work for implemented flows.
- ■ Resume processing uses validated AI output and errors are handled consistently.
- ■ At least one end-to-end resume workflow works across frontend and backend.
- ■ Tests cover principal success/failure cases; configuration and limitations are documented.
- ■ Frontend collaborator can integrate without guessing the API contract.

## 15. Immediate Next Steps for the Backend Collaborator

- Audit the existing backend and identify working versus incomplete routes/models/services.
- Share a confirmed API inventory before adding duplicate endpoints.
- Verify FastAPI startup and database configuration.
- Complete authentication and candidate profile access.
- Prioritize resume upload → processing → analysis retrieval.
- Agree on the AI extraction schema with the AI/ML collaborator.
- Add career insights and roadmap APIs based on available logic and stored results.
- Test frontend integration and document limitations/follow-up tasks.

### Expected handoff from backend collaborator

- Backend startup instructions and confirmed routes/methods.
- Request/response examples and authentication requirements.
- Database/migration instructions and environment variable names.
- AI processing input/output contract.
- Test results and known issues.

**Final objective: Deliver a stable FastAPI layer connecting the Road2Hire frontend, database, and available AI/ML modules through documented, validated, and testable APIs.**